

Chapter 6: Problem-Solving Agents, Problem Formulation and Search Strategies

Student Class Notes with Mathematical Foundations, Solved Examples and Practice Questions

Artificial Intelligence | M.Sc. Mathematics with AI & Data Science | CSJMU, Kanpur

6.1 Learning Objectives

- Understand the role of search in artificial intelligence.
- Describe the operation of a problem-solving agent.
- Formulate well-defined search problems.
- Represent state spaces using sets, functions and graphs.
- Distinguish tree search from graph search.
- Compare uninformed and informed search strategies.
- Apply BFS, DFS, UCS, Greedy Best-First and A* to small examples.
- Analyze completeness, optimality, time and space requirements.

6.2 Introduction

An intelligent agent does not always know the correct next action immediately. In many tasks, it must examine possible future states before choosing what to do. This computational process is called search. A problem-solving agent is a goal-based agent that searches for a sequence of actions leading from an initial state to a goal state.

Examples include route planning, puzzle solving, robot navigation, scheduling, game playing, theorem proving and network routing. Although the applications differ, their mathematical structure is often the same: states are vertices, legal actions are edges, and a solution is a path.

6.3 Problem-Solving Agent

A problem-solving agent follows a cycle:

1. Perceive the current situation and update the internal state.
2. Formulate a goal.
3. Formulate a search problem.
4. Run a search algorithm to obtain a solution path.
5. Execute actions from the solution; replan when required.

The output of search is generally an action sequence, not merely a final state.

6.4 Goal Formulation and Problem Formulation

Goal formulation decides what the agent wants to achieve. Problem formulation decides which details of the world must be represented so that a search procedure can find a way to achieve it.

Stage	Question	Example
Goal formulation	What outcome is required?	Reach the destination.
Problem formulation	What states, actions, transitions and costs matter?	Model the road network.

Stage	Question	Example
Search	Which action sequence reaches the goal?	Explore routes.
Execution	How is the solution carried out?	Follow the selected roads.

6.5 Mathematical Definition of a Search Problem

A well-defined search problem may be written as:

$$P = \langle S, A, \text{Result}, s_0, \text{Goal}, c \rangle$$

- S is the state space.
- $s_0 \in S$ is the initial state.
- $A(s)$ is the set of actions applicable in state s .
- Result: $S \times A \rightarrow S$ is the transition function.
- Goal: $S \rightarrow \{\text{True}, \text{False}\}$ is the goal-test predicate.
- $c(s, a, s') \geq 0$ is the step-cost function.

For an action sequence $\pi = \langle a_1, \dots, a_k \rangle$ producing states $s_0, s_1, \dots, s_k$, the path cost is

$$C(\pi) = \sum_{i=0}^{k-1} c(s_i, a_{i+1}, s_{i+1}).$$

A solution satisfies $\text{Goal}(s_k) = \text{True}$. An optimal solution π^* minimizes $C(\pi)$ among all solutions.

6.6 State-Space Graph

A state-space graph is $G = (V, E)$, where V is the set of states and E contains directed transitions. A weighted graph also has a weight function $w: E \rightarrow \mathbb{R}_{\geq 0}$. Search is therefore related to graph theory, combinatorics and optimization.

- Vertex = state.
- Edge = legal action or transition.
- Path = sequence of actions.
- Path length = number of edges.
- Path cost = sum of edge costs.
- Branching factor b = average or maximum number of successors.
- Solution depth d = number of steps in the shallowest solution.

6.7 State versus Search Node

State	Search Node
Represents a configuration of the problem world.	A data record used by the search algorithm.
Example: a particular tile arrangement.	Contains state, parent, generating action, path cost and depth.
The same state is conceptually one configuration.	Several nodes may refer to the same state through different paths.

6.8 Tree Search and Graph Search

Tree search may generate the same state repeatedly. Graph search maintains an explored set (closed set) and often a frontier set (open set), reducing repeated work and preventing many cycles.

Term	Meaning
Frontier	Generated but not yet expanded nodes
Explored set	States already expanded
Expand	Apply all legal actions to generate successors
Goal test	Check whether a selected or generated state satisfies the goal
Solution reconstruction	Follow parent links from a goal node to the root

6.9 Problem Examples

6.9.1 Route-Finding Problem

Component	Description
State	Current location
Initial state	Starting location
Actions	Travel along connected roads
Transition	Move to the endpoint of the chosen road
Goal test	Destination reached
Cost	Distance, time, toll, fuel or weighted cost

6.9.2 8-Puzzle

The 8-puzzle contains eight numbered tiles and one blank in a 3×3 grid. An action moves the blank to an adjacent position. With a fixed goal configuration, only half of the 9! permutations are reachable, giving $9!/2 = 181,440$ reachable states.

- Mathematics: permutations and parity.
- Graph theory: each configuration is a vertex.
- Optimization: seek a minimum-move solution.
- Heuristics: misplaced tiles or Manhattan distance.

6.9.3 Water-Jug Problem

For jugs of capacities 4 and 3 litres, a state may be written (x, y) , where $0 \leq x \leq 4$ and $0 \leq y \leq 3$. Actions include fill, empty and pour. The goal may be $x = 2$ or $y = 2$. This example shows how continuous real-world actions can be abstracted into a finite discrete state space.

6.10 Evaluation of Search Strategies

- Completeness: finds a solution if one exists.
- Optimality: finds a minimum-cost solution.
- Time complexity: number of generated or expanded nodes.
- Space complexity: maximum number of stored nodes.

Symbols commonly used: b (branching factor), d (shallowest solution depth), m (maximum depth), ℓ (depth limit), C^* (optimal cost), ϵ (minimum positive step cost).

6.11 Uninformed Search Strategies

Uninformed search uses no estimate of closeness to the goal. Its knowledge is limited to the formal problem definition.

6.11.1 Breadth-First Search

- Expands the shallowest frontier node.
- Frontier structure: FIFO queue.
- Complete for finite branching factor.
- Optimal for equal step costs.
- Time and space grow exponentially, approximately $O(b^d)$.

BFS is appropriate when the solution is expected to be shallow, actions have equal cost and memory is available.

6.11.2 Depth-First Search

- Expands the deepest frontier node.
- Frontier structure: LIFO stack.
- Space approximately $O(bm)$, much lower than BFS.
- Not generally complete in infinite spaces and not optimal.
- Useful when memory is limited and solutions may be deep.

6.11.3 Uniform-Cost Search

- Expands the node with minimum path cost $g(n)$.
- Frontier structure: priority queue ordered by $g(n)$.
- Complete if step costs are bounded below by $\epsilon > 0$.
- Optimal for non-negative step costs.
- Equivalent in expansion order to BFS when all step costs are equal.

6.11.4 Depth-Limited Search

Depth-limited search is DFS that does not expand nodes below limit ℓ . It avoids infinite descent, but a goal deeper than ℓ will not be found.

6.11.5 Iterative Deepening Search

Iterative deepening repeatedly applies depth-limited search with limits 0, 1, 2, It is complete for finite b , optimal under unit step costs, and requires only linear memory in the solution depth.

6.12 Informed Search Strategies

Informed search uses a heuristic $h(n)$, an estimate of the cost from n to a goal. A useful heuristic reduces the number of expanded nodes by directing search toward promising regions.

6.12.1 Greedy Best-First Search

Evaluation function: $f(n) = h(n)$.

- Selects the apparently closest node to the goal.
- May be fast with a good heuristic.
- Does not account for the cost already paid.
- Not generally optimal.

6.12.2 A* Search

Evaluation function: $f(n) = g(n) + h(n)$.

A* combines actual path cost and estimated remaining cost. A heuristic is admissible when $0 \leq h(n) \leq h^*(n)$, where $h^*(n)$ is the true optimal remaining cost. A consistent heuristic satisfies

$$h(n) \leq c(n, a, n') + h(n')$$

With suitable conditions, A* is complete and optimal. However, it may use large amounts of memory.

6.13 Heuristic Mathematics for the 8-Puzzle

Heuristic	Definition	Observation
Misplaced tiles	Number of numbered tiles not in goal position	Simple lower bound
Manhattan distance	$\sum \text{current row} - \text{goal row} + \text{current column} - \text{goal column} $	Usually more informative
Perfect heuristic	True remaining optimal cost $h^*(n)$	Ideal but as hard as solving the problem

If $h_2(n) \geq h_1(n)$ for all n and both are admissible, h_2 dominates h_1 and generally allows A* to expand no more nodes than h_1 , apart from tie-handling details.

6.14 Comparison of Major Strategies

Method	Priority	Complete	Optimal	Space
BFS	Depth	Yes*	Yes for unit cost	High
DFS	Deepest	No in general	No	Low
UCS	$g(n)$	Yes*	Yes	High
DLS	Deepest to ℓ	Conditional	No	Low
IDDFS	Increasing ℓ	Yes*	Yes for unit cost	Low
Greedy	$h(n)$	Not in general	No	Often high
A*	$g(n)+h(n)$	Yes under conditions	Yes with suitable h	High

*Under standard assumptions such as finite branching and positive step costs where relevant.

6.15 Solved Example

Graph edges: $S \rightarrow A(1)$, $S \rightarrow B(4)$, $A \rightarrow C(2)$, $A \rightarrow D(5)$, $B \rightarrow D(1)$, $C \rightarrow G(4)$, $D \rightarrow G(1)$.

- Path $S-A-C-G$ has cost $1+2+4 = 7$.
- Path $S-A-D-G$ has cost $1+5+1 = 7$.
- Path $S-B-D-G$ has cost $4+1+1 = 6$.
- All three paths have depth 3, so BFS cannot distinguish them by cost; child order matters.
- UCS returns $S-B-D-G$ because it has the minimum total cost 6.

6.16 Applications

- GPS and network route planning
- Robot navigation and warehouse automation

- Scheduling and timetabling
- Puzzle and game solving
- Automated theorem proving
- Planning actions in software agents
- Bioinformatics sequence and structure search
- Cryptographic search problems, key-space reasoning and attack modelling

6.17 Key Takeaways

- Problem-solving agents search before acting.
- A precise problem formulation is essential; a poor abstraction can make search inefficient or incorrect.
- Search is path finding in a state-space graph.
- BFS focuses on depth, UCS on path cost, DFS on memory efficiency, and A* on cost plus heuristic guidance.
- Completeness and optimality are different from practical efficiency.
- Mathematics provides the language of sets, functions, graphs, combinatorics, optimization and complexity.

6.18 Multiple-Choice Questions

1. A problem-solving agent primarily searches for:

- A. A percept
- B. An action sequence leading to a goal
- C. A sensor
- D. A database

Answer: B

2. Which is not a component of a standard search problem?

- A. Initial state
- B. Transition model
- C. Goal test
- D. Neural-network loss

Answer: D

3. BFS uses a:

- A. Stack
- B. FIFO queue
- C. Priority by $h(n)$
- D. Random queue

Answer: B

4. DFS is attractive mainly because of its:

- A. Guaranteed optimality
- B. Low memory requirement
- C. Use of heuristic
- D. Unit cost

Answer: B

5. UCS expands the node with minimum:

- A. Depth
- B. $h(n)$
- C. $g(n)$
- D. Number of children

Answer: C

6. Greedy best-first search uses:

- A. $f=h$
- B. $f=g$
- C. $f=g+h$
- D. $f=g-h$

Answer: A

7. A* uses:

- A. Only past cost
- B. Only estimated cost
- C. Past cost plus estimated remaining cost
- D. Depth only

Answer: C

8. An admissible heuristic:

- A. Always overestimates
- B. Never overestimates true remaining cost
- C. Must be exact
- D. Must be negative

Answer: B

9. BFS is optimal when:

- A. Step costs are equal
- B. The tree is infinite
- C. Memory is zero
- D. Heuristic is perfect

Answer: A

10. A solution in search is normally:

- A. A single percept
- B. A sequence of actions
- C. Only a goal state label
- D. A sensor reading

Answer: B

6.19 Short-Answer Questions

11. Define a problem-solving agent.
12. Differentiate goal formulation and problem formulation.
13. What is a state space?

14. Write the mathematical components of a search problem.
15. Differentiate state and node.
16. What is the frontier in graph search?
17. State two advantages and two limitations of BFS.
18. Why is DFS not optimal?
19. Define an admissible heuristic.
20. Explain $g(n)$, $h(n)$ and $f(n)$ in A^* .

6.20 Long-Answer and Practice Problems

21. Formulate the 8-puzzle as a search problem and explain its mathematical structure.
22. Compare BFS, DFS and UCS with respect to completeness, optimality, time and space.
23. Explain A^* search and the role of admissible and consistent heuristics.
24. Formulate the water-jug problem using state variables, operators, goal test and cost.
25. For a campus map of your choice, create a weighted graph and trace BFS and UCS.
26. Design a heuristic for a grid-navigation problem containing obstacles. Explain why it is or is not admissible.
27. Discuss how the choice of abstraction changes the size of a state space.
28. Explain why repeated-state detection is important in graph search.

6.21 Answer Key

MCQs: 1-B, 2-D, 3-B, 4-B, 5-C, 6-A, 7-C, 8-B, 9-A, 10-B.

6.22 References

29. Russell, S. J. and Norvig, P., Artificial Intelligence: A Modern Approach, 4th ed., Chapter 3.
30. AIMA official chapter, algorithms and figures resources, University of California, Berkeley.
31. MIT OpenCourseWare, Artificial Intelligence and Algorithms search lectures.